

AUDIT

Environment-adaptability audit

Operator directive that produced this document, verbatim:

“Make sure all this is fully dynamic and adaptable based on the environment!”

It was issued about one specific proposal, but it generalises: this repository should carry no frozen assumption about the machine it runs on. This file is the audit. The enforcement is `scripts/audit-environment-assumptions.sh`.

Audit date: **2026-08-31**. Scope **as audited on that date**: the umbrella repository’s own tracked files only.

That scope statement is now historical, and every count in §4.1–§4.3 below belongs to it. On 2026-09-01 the gate’s gitlink blind spot was closed and the sweep was extended over the whole declared fleet — 1 762 files across 14 repositories, measured that morning; **1 802 files by the evening of the same day**, which is the figure the block immediately below carries and the one §4.8’s 1 762 should be read against. The findings that expansion surfaced are triaged in **§4.8**, and they are not comparable to the §4.1 numbers because they do not describe the same population. Do not add the two together.

Re-derived state — 2026-09-02

Two things happened here, and only one of them is a fix. **The stale rules were deleted; F13 and F14 were fixed in source and their baselines then removed.** Removing a baseline without fixing the defect would have moved debt between columns and changed nothing the gate can see, so the order matters and is recorded: source first, ledger second.

```

bash scripts/audit-environment-assumptions.sh #
rc 0 before AND after
bash scripts/audit-environment-assumptions.sh --strict-allow-
list # rc 1 -> rc 0

```

measured	before	after
exit code	0	0
--strict-allow-list exit code	1	0
files scanned	2 166	2 166
repositories	14	14
baselined occurrences	728	683, across 216 files
justified (allow-listed) occurrences	531	533
allow rules, total	444	436 — 396 external + 40 embedded
stale allow rules	2 of 444	0 of 436
PATH-ABSENT allow rules	0	0
third-party NOTE	1	1

The +2 on the justified row is NOT this change. It is `scripts/lumen-index-doctor.sh`, edited concurrently by another agent while this work ran (`git diff --stat` on it: 229 insertions), which took its existing `ENDPOINT REASON` rule from 3 matches to 5. Attributed rather than absorbed (§11.4.6). Everything else on that table is accounted for below, exactly: **-45 baselined occurrences and -8 allow rules (6 + 2).**

The 2 stale rules are DELETED — the owed work the 2026-09-01 block below recorded as “recorded here rather than done”. Each was re-derived by hand before deletion, not merely taken from the gate’s own report:

was at rule-file line	rule	why it was genuinely stale
10	<code>scripts/audit-hardcoded-paths.sh * * (REASON)</code>	Of the twelve class patterns, exactly ONE matches anywhere in that file — an <code>OSPATH</code> hit at line 38 — and that line is a <code>#</code>

was at rule-file line	rule	why it was genuinely stale
134	<code>_tools/gen/ review_ui_all.py</code> <code>MODEL *</code> (BASELINE, cited F15)	comment, which the scanner blanks for <code>.sh</code> before any class is tested. Every other class pattern matches zero times and the shebang is already <code>/usr/bin/env</code>. It suppressed nothing while standing as a blanket <code>* *</code> exemption over a whole file. The file still exists and is still scanned. Its only MODEL-class token is <code>groq/</code> llama-3.3-70b on line 3, inside the module docstring; the MODEL class requires a quote character on the same line before it counts a model id as code rather than prose, and that line carries none. <code>git log -p</code> over the file's whole history shows that docstring line is the only occurrence it has ever had — there was never a bare module constant

was at rule-file line	rule	why it was genuinely stale
		— which is why the §4.4 F15 file list never named it.

F13 and F14 are CLOSED, in source. `_tests/env.js` is new and is the single place the harness learns where to bind and what to request: `VD_PORT/MV_PORT`, `VD_BASE/MV_BASE`, `MOTION_VD_PORT/MOTION_MV_PORT` + bases, and `UI_L10N2_PORT/UI_L10N2_BASE`, each `process.env`-derived with the old literal as the documented default. Both Playwright configs, all 17 affected specs and both standalone drivers read it. That also closes the half of F13/F14 that no allow rule could express: the port a config **bound** and the base a spec **requested** were two independent literals free to disagree, and a disagreement surfaced as an assertion about the *site*. They are now one value.

Measured, not asserted:

```
cd _tests
npx playwright test --project=chromium tests/vasic-
digital.spec.js          # 12 passed
VD_PORT=9401 MV_PORT=9082 npx playwright test --project=chromium \
  tests/vasic-
digital.spec.js          # 12
passed
VD_PORT=9401 MV_PORT=9082 npx playwright test --project=chromium \
  tests/seo-meta.spec.js tests/ui-l10n-
chrome.spec.js          # 41 passed
MOTION_VD_PORT=9481 MOTION_MV_PORT=9482 node tools/motion-
audit.cjs              # "servers up"
UI_L10N2_PORT=9791 node ui-l10n2-
verify.js              # rc 0, 4 pages
```

Running on non-default ports was **impossible** before this change; that is the whole content of F13. The `motion-audit.cjs` run stops later at `browserType.launch: Executable doesn't exist ... firefox-1532` — a browser that is not installed on this host, unrelated and pre-existing, reported so it is not mistaken for a consequence.

Independent proof that the source, not the ledger, is what changed.

The HEAD version of the gate — the one that still carries all six F13/F14 baseline rows — run against the FIXED tree reports **8 stale allow rules**: the 2 above plus the 6 whose defects no longer exist. That is precisely what a fix looks like from a gate's point of view, and precisely why leaving the rows behind would have been worthless:

```
git show HEAD:scripts/audit-environment-assumptions.sh > /tmp/orig.sh
bash /tmp/orig.sh . --strict-allow-list      # rc 1 – "8 STALE allow rule(s)"
bash scripts/audit-environment-assumptions.sh --strict-allow-list # rc 0
```

And `_tests/env.js` itself introduces nothing. It is **untracked** at this writing, so `git ls-files` does not show it to the gate; it was therefore audited separately, in a synthetic checkout containing the 28 changed/added `_tests` files and **no allow rule matching any of them**: `bash scripts/audit-environment-assumptions.sh <sandbox> → rc 0`, “*no NEW frozen environment assumptions*”, 28 files scanned, zero suppressions used. Until those files are committed, the umbrella run’s 2 166-file count does not include `env.js`.

`bash scripts/audit-environment-assumptions.sh --prove-failure` still exits **0** at 13 of 13 mutations after the edits to the gate’s own allow-list.

683 is still debt. The gate is green and green still does not mean clean. **And 683 did not survive the day** — see the F15 slice immediately below, which took it to **666**.

F15 — CLOSED in source, 2026-09-02 (second slice of the same day)

Same order as F13/F14, for the same reason: **source first, ledger second**. Removing a baseline without fixing the defect moves debt between columns and changes nothing the gate can see.

The 17 rows, enumerated before anything was touched. The F15 row in §4.4 said “15 occurrences” and its file list named nine files summing to 15. The gate measures **17**, and the two extra are accounted for exactly: they are the §4.6 rows 12 and 13 docstrings (`translate_ui_chunked.py:2`, `translate_home.py:312`), which the blanket `<file> MODEL * baselines` swallowed alongside the real defects. The 15 was never a measurement of what the rules suppressed. The command that produced the list — the live gate with the nine F15 rules stripped from a scratch copy of it, so nothing else changed:

```
cp scripts/audit-environment-assumptions.sh /tmp/enum.sh
# delete the nine `_tools/gen/*.py MODEL *` rules and their marker comments
bash /tmp/enum.sh --no-submodules "$PWD"
```

file	line	the frozen literal
_tools/gen/ repair_ui_terms.py	63	-provider", "zhipu", "-model", "glm-4.5- flash"
_tools/gen/translate- ui.py	86	"-provider", "zhipu", "-model", "glm-4.5-flash"
_tools/gen/ translate_aria_footer.py	23	TRANSLATOR = "zai- glm-4.7"
_tools/gen/ translate_aria_footer.py	25	REVIEWER = "gpt- oss-120b"
_tools/gen/ translate_aria_footer.py	88	if model == "gpt- oss-120b":
_tools/gen/ translate_home.py	50	GROQ_TRANSLATOR = "llama-3.3-70b- versatile"
_tools/gen/ translate_home.py	51	CEREBRAS_TRANSLATOR = "gpt-oss-120b"
_tools/gen/ translate_home.py	52	REVIEWER = "gpt- oss-120b"
_tools/gen/ translate_home.py	312	function docstring naming gpt-oss-120b
_tools/gen/ translate_ui_all.py	81	"-provider", "zhipu", "-model", "glm-4.5-flash"
_tools/gen/ translate_ui_batch.py	88	"-provider", "zhipu", "-model", "glm-4.5-flash"
_tools/gen/ translate_ui_chunked.py	2	module docstring naming glm-4.5- flash
_tools/gen/ translate_ui_chunked.py	60	"-model", "glm-4.5- flash"
_tools/gen/ translate_ui_headroom.py	21	TRANSLATOR = "llama-3.3-70b- versatile"
_tools/gen/ translate_ui_headroom.py	22	REVIEWER = "gpt- oss-120b"
	77	

file	line	the frozen literal
_tools/gen/ translate_ui_headroom.py		if model == "gpt-oss-120b":
_tools/gen/ translate_ui_slow.py	47	-provider", "zhipu", "-model", "glm-4.5- flash"

What replaced them. Every id is now `os.environ.get(...)` with the former literal as the documented default, so an unset environment is byte-identical to the previous behaviour. **No new file was created**, deliberately: the gate enumerates with a bare `git ls-files`, so an untracked shared helper would be invisible to it (that is exactly what happened to `_tests/env.js` in the F13/F14 slice). Every change lives in an already-tracked file and is therefore fully scanned.

key	default (the former literal)	used by
UI_TRANSLATOR_PROVIDER	zhipu	the six HelixTranslate engine drivers
UI_TRANSLATOR_MODEL	glm-4.5-flash	repair_ui_terms, translate-ui, translate_ui_all, translate_ui_batch, translate_ui_chunked, translate_ui_slow
UI_HEADROOM_TRANSLATOR_MODEL	llama-3.3-70b-versatile	translate_ui_headroom.py
UI_HEADROOM_REVIEWER_MODEL	gpt-oss-120b	translate_ui_headroom.py
ARIA_FOOTER_TRANSLATOR_MODEL / _PROVIDER	zai-glm-4.7 / cerebras	translate_aria_footer.py
ARIA_FOOTER_REVIEWER_MODEL / _PROVIDER	gpt-oss-120b / cerebras	translate_aria_footer.py
HOME_GROQ_TRANSLATOR_MODEL	llama-3.3-70b-versatile	translate_home.py
HOME_TRANSLATOR_MODEL / HOME_TRANSLATOR_PROVIDER	gpt-oss-120b / cerebras	translate_home.py
HOME_REVIEWER_MODEL / HOME_REVIEWER_PROVIDER	gpt-oss-120b / cerebras	translate_home.py
UI_REASONING_EFFORT_MODELS	gpt-oss-120b	

key	default (the former literal)	used by
		translate_aria_footer.py, translate_ui_headroom.py

Three decisions in that table are worth stating rather than leaving to be inferred:

- **Per-pipeline key spaces, not one shared TRANSLATOR_MODEL.** §4.4's remediation column suggested reusing the shell pipeline's names. That would have introduced cross-talk: `_tools/translate/translate-content.sh` already exports `TRANSLATOR_MODEL=mistral-large-latest` for the *document* batch, and a UI driver reading the same key would silently follow it onto a different provider. The `UI_*` prefix is not invented here — `UI_KEY`, `UI_BASEURL`, `UI_WORKERS`, `UI_GAP`, `UI_BUDGET`, `UI_PASSES` already exist in these very files.
- **The provider moved with the model.** A model override that leaves `-provider zhipu` frozen is a half-fix that fails on the first substitution, so each provider literal adjacent to a derived model became derived too, with its own former literal as the default. The `PROVIDER` class is not one the gate detects; this is wider than the finding, on purpose.
- **`if model == "gpt-oss-120b"` became a membership test.** That comparison selects the OpenAI-style `reasoning_effort` knob — a *capability* of a model family, not a choice of model — so freezing it would have re-broken the fix the moment a substitute model was supplied. It reads `UI_REASONING_EFFORT_MODELS` (comma-separated, default `gpt-oss-120b`).
- **The two docstrings were reworded, not allow-listed.** §4.6 rows 12 and 13 already judged them non-defects, so a `# REASON: row` would have been defensible — and it would still have been debt moved between columns. They now name the variable instead of the literal, which is a real change and is why the count falls by 17 rather than 15.

Measured, not asserted. Every one of the nine files byte-compile, and the env layer was exercised in both directions:

```
python3 -m py_compile _tools/gen/{repair_ui_terms,translate-
ui,translate_aria_footer,\
translate_home,translate_ui_all,translate_ui_batch,translate_ui_ch
\
translate_ui_headroom,translate_ui_slow}.py
# rc 0
```


With every F15 variable **unset**, each module resolves to the exact former literal — PROVIDER='zhipu' MODEL='glm-4.5-flash' in all six engine drivers; TRANSLATOR='llama-3.3-70b-versatile' REVIEWER='gpt-oss-120b' in translate_ui_headroom; TRANSLATOR='zai-glm-4.7' ... REVIEWER='gpt-oss-120b' in translate_aria_footer; GROQ_TRANSLATOR='llama-3.3-70b-versatile' CEREBRAS_TRANSLATOR='gpt-oss-120b' REVIEWER='gpt-oss-120b' in translate_home. With them **set**, every constant follows the environment. The argv itself was checked, not just the constant: with HELIX_BIN pointed at a stub that records its arguments, repair_ui_terms.call, translate_ui_chunked.call_engine and translate_ui_slow.call each emitted

```
-provider zhipu -model glm-4.5-flash          # variables unset
-provider probe-prov -model probe-model      #
UI_TRANSLATOR_PROVIDER/_MODEL set
```

Independent proof that the source, not the ledger, is what changed.

The HEAD version of the gate — which still carries all ten _tools/gen/*.py MODEL rules — run against the FIXED tree reports every one of them as **STALE**, “**path IS scanned, rule matched nothing**”:

```
git show HEAD:scripts/audit-environment-assumptions.sh > /tmp/orig.sh
bash /tmp/orig.sh --no-submodules --stale-rules "$PWD"
# STALE – path IS scanned, rule matched nothing: 17
# BASELINE _tools/gen/translate_home.py MODEL * ... and
the other nine
```

(Ten rules, not nine: review_ui_all.py was already stale at HEAD and was deleted earlier the same day — see the table in the F13/F14 block above.)

measured	before	after
audit-environment-assumptions.sh exit code	1	1
--strict-allow-list exit code	1	1
audit-hardcoded-paths.sh exit code	0	0
files scanned	2 166	2 166
repositories	14	14
baselined occurrences	683	666
embedded allow rules	40	31
stale allow rules	0	0
frozen environment assumptions (findings)	6	7

measured	before	after
--prove-failure	13/13, rc 0	13/13, rc 0

Two rows on that table are NOT this change, and saying so is the point.

1. **The gate exits 1 in both columns, and it did before this slice started.** The findings are scripts/verify-all-constitution-rules.sh (3 × GITREF, lines 615/619/629) and workshop/platform/backend/cmd/workshop-server/main.go (MODEL), both outside _tools/gen/ and both outside the scope of this change. The “rc 0” the F13/F14 block above records was true when written; it is not true now, and this slice did not cause the difference.
2. **The findings count moved 6 → 7 while this ran.** The seventh is workshop/platform/backend/cmd/workshop-server/main.go:1078, if model == "nomic-embed-text" && — a line that appeared in that file’s working tree during this work (git -C workshop diff --stat on it: 184 insertions, 12 deletions against HEAD). It is a concurrent agent’s edit in a submodule this change never touched. Attributed rather than absorbed (§11.4.6). **After this slice, _tools/gen/ contributes zero findings and zero baselined occurrences** — grep -c _tools/gen over the full report returns 0.

The external .environment-assumptions-allow moved from **391** rules at HEAD to **398** in the working tree over the same window. That file is modified in the working tree, but **not by this change** — no edit in this slice touched it, and the +7 rules are another agent’s concurrent work on the workshop benchmark MODEL rows. The **-9 embedded** rules (scripts/audit-environment-assumptions.sh: 40 → 31) are entirely this one’s. Counted with:

```
grep -vE '^[[:space:]]*(#|$)' .environment-assumptions-allow | wc
-l          # 398
awk '/^ALLOW_RULES="\$(cat <</{f=1;next} /^ALLOW_EOF$/{f=0} f' \
scripts/audit-environment-assumptions.sh | grep -vE
'^[[:space:]]*(#|$)' | wc -l    # 31
```

666 is still debt. The umbrella-root-owned share of it is now **28**, measured rather than inferred:

```
bash scripts/audit-environment-assumptions.sh --no-submodules
# 28 baselined occurrence(s), 186 file(s) across 1 repository
(was 45 before this slice)
```

So **638 sit inside submodules** and can only be fixed by commits in the repositories this umbrella consumes, returning as gitlink bumps.

Re-derived state — 2026-09-01, late

SUPERSEDED IN PART, 2026-09-02 — see the block above. Specifically: the “stale allow rules 2 of 401” row, and the sentence below stating that deleting them “is owed work and is recorded here rather than done”, are **WITHDRAWN**; both rules are gone. The baselined-occurrence figure in this block is likewise superseded twice over (708 → 728 → 683). The rest stands as the dated observation it was.

Everything below this block was written earlier the same day and had drifted in the direction that matters least: it **understated** what had been fixed. It is corrected in place rather than appended to, and the corrections are marked.

```
bash scripts/audit-environment-assumptions.sh # rc 0
```

measured	value
exit code	0
files scanned	1 802
repositories	14
classes	12
baselined occurrences	708, across 228 files
justified (allow-listed) occurrences	464
allow rules, total	401 — 353 in .environment-assumptions-allow, 48 embedded in the script
stale allow rules	2 of 401
third-party NOTE	1, out of scope per §11.4.29

The gate is **green**, and **green here does not mean clean**: 708 baselined occurrences are real, known, unfixed defects that the gate prints on every run by design. A baseline is recorded debt, never a justification.

The 2 stale rules are both in the script’s **embedded** ALLOW_RULES heredoc, not in the external file:

rule-file line	rule
10	scripts/audit-hardcoded-paths.sh * * (REASON)

rule-file line	rule
134	<code>_tools/gen/review_ui_all.py</code> MODEL * (BASELINE, cites F15)

Both are stale in the sense the gate means: the **occurrence** each names is gone while the exemption still stands **at that path**. Neither file was deleted — `_tools/gen/review_ui_all.py` still exists — so this is exemption rot, not a dead path. Note also that the §4.4 F15 file list does not name `review_ui_all.py`, so that BASELINE cites a finding id that does not cover it. Both rules live in `scripts/audit-environment-assumptions.sh`, which the change that re-derived this document was not permitted to edit; deleting them is owed work and is recorded here rather than done.

The third-party NOTE is `submodules/superspec/.github/workflows/ci.yml:21` — `python-version: "3.12"`. **§4.6 row 19 gives the path as `.specify/extensions/superspec/...`. That is not where the gate reports it;** the finding is the same upstream pin and the out-of-scope reasoning is unchanged, but the path in that row is wrong and is corrected there.

What is NOT claimed here. `go test -race` was **not** run against `submodules/LLMProvider` while re-deriving this — only `-short`. No statement about data races in that module appears anywhere in this document, and none may be added without running it.

1. What this covers, and what it does not

`scripts/audit-hardcoded-paths.sh` already exists and catches exactly one class: machine-specific **absolute paths** (`<macos-host>/...`, `/home/<someone>/...`). It is blind to everything else that ties the tree to one box.

`scripts/audit-environment-assumptions.sh` is its sibling for the broader class. The two are complementary and both must pass; neither subsumes the other.

The failure mode being hunted is **not** a loud crash. It is silent misbehaviour — the shape this repository has already been burned by twice (a CI gate that reported success over an empty build; a test that stayed green while the thing it tested had been deleted). Three concrete instances found here:

Frozen assumption	What actually happens elsewhere
systemctl show + journalctl read the Ollama backend	On a host without systemd both return empty through <code>2>/dev/null, backend_library()</code> yields "", and the Vulkan refusal that the function exists to trigger never fires . The script reports success while doing none of its safety work.
/Applications/Open Design.app	On Linux the <code>-x</code> test fails, the daemon never starts, and every downstream diagram generation silently produces nothing.
/etc/sysconfig/ollama	On a Debian-family host the file systemd reads is <code>/etc/default/ollama</code> . The write “succeeds” into a file nothing reads.

Because of that, `2>/dev/null` is **not treated as a guard by this audit**. It is the delivery mechanism for the bug.

2. Scan universe

	Count
Paths in git ls-files	5,768
Excluded: generated evidence (<code>_tests/evidence/</code>), English + 14 translated content trees (<code>_content*</code>), analysis and prose (<code>_analysis/</code> , <code>docs/</code>), binaries, lockfiles, <code>node_modules/vendor</code> , and submodule gitlinks	5,594
	174

Count
Scanned (executable + configuration source)

`bash scripts/audit-environment-assumptions.sh --list` prints the exact list.

Excluding prose is not a loophole — it is the false-positive rule this project learned the hard way, when a raw `grep` count of 187 was reported as a finding and 167 of the hits turned out to be legitimate token definitions. A literal in a comment, a docstring, a documented example, a test fixture, or a non-translatable-term list configures nothing and is not a defect.

3. Classification rule

Every class-pattern hit was put through one question:

Is there an escape hatch that lets a different machine choose a different value, without editing the file?

If yes, the literal is a **default**, not an assumption, and is not a defect. Accepted escape hatches:

Kind	Form
shell env default	<code>HOST="\${OLLAMA_HOST:-http://localhost:11434}"</code>
python	<code>os.environ.get("UI_WORKERS", "5")</code> , or a local <code>env()</code> wrapper over <code>os.environ</code>
node	<code>process.env.VD_BASE \\ </code> <code>'http://localhost:8401'</code>
go	<code>os.Getenv("...")</code>
capability probe	<code>command -v systemctl</code> , <code>check_command brew</code> , type <code>-p X</code>
OS dispatch	<code>case "\$(uname -s)", \$OSTYPE</code> , <code>process.platform</code> , <code>sys.platform</code> , <code>runtime.GOOS</code>
existence probe	<code>[[-d /etc/sysconfig]]</code> , or <code>listing /etc/sysconfig/x /etc/</code>

Kind	Form
	default/x /etc/conf.d/x together
declared fallback	a variable literally named fallback / default / DEFAULT_*, i.e. the last term of a precedence chain

Escape hatches apply to the **value** classes only. No environment variable can make `sed -i` portable, so GNUBSD, SHEBANG, SERVICE, PKGMGR and GITREF are never cleared by an env override — only by a capability probe or OS dispatch (and GNUBSD only by an actual dual-form call).

Severity

Severity	Definition
HIGH	Silently produces a wrong or missing result on another machine, with no error surfaced to the caller.
MEDIUM	Breaks or misbehaves elsewhere, but loudly, recoverably, or only under contention — including a wrong pass rather than a wrong artefact.
LOW	Degrades, drifts, or is cosmetic; overridable by a flag, or non-fatal by construction.

4. Results

4.1 Headline numbers

Every number below is a count of **things**, each of which is listed. None is a `grep -c` line count.

	Count
Files scanned	174
	51

	Count
Files with at least one class-pattern hit	
Raw class-pattern occurrences	104
Verified defects	85 (in 46 files)
False positives (hand-checked, not defects)	19 (5 files are pure false positives)
Classes that fired	9 of 12
Classes that fired zero times	SERVICE, GPU, GITREF

SERVICE and GPU fired zero times **because they were fixed by a concurrent agent during this audit** — see §4.5. They were live findings when the sweep started. GITREF has genuinely never had an instance in this repository.

4.2 By class

Class	Raw hits	Defects	False positives
ENDPOINT	51	46	5
MODEL	24	15	9
GNUBSD	8	6	2
OSPATH	7	5	2
HOSTNAME	6	6	0
TOOLVER	4	3	1
PKGMGR	2	2	0
SHEBANG	1	1	0
PARALLEL	1	1	0
SERVICE	0	0	0
GPU	0	0	0
GITREF	0	0	0
Total	104	85	19

4.3 By severity

Severity	Occurrences	Findings
HIGH	10	F10, F11, F12
MEDIUM	53	F3, F6, F7, F8, F9, F13, F14, F16, F19

Severity	Occurrences	Findings
LOW	22	F15, F18, F20, F22

4.4 Findings

Line numbers are a **2026-08-31 snapshot**. Several of these files were being edited by other agents while the sweep ran, so line numbers drift; the gate’s allow-list is keyed on path + class + literal substring, never on line number.

ID	Sev	File : line	Literal	Why port
F3	MED	scripts/ollama-vulkan-remediation.sh:429	sudo sed -i "\ ^\${FLAG}\$\ d" "\$ENVFILE"	GNU BSD, cons argu suffi the v erro Sam insic mut so th itsel mut BSD, has n emp dev/ GNU sort. so th back and 0.0.
F6	MED	scripts/verify-governance-cascade.sh:708	sed -i 's/^ - name: monetization\$/. ./'	GNU mac stat mod siler
F7	MED	scripts/setup-agents-wizard.sh:1507	readlink -f "\$_lb"	
F8	MED	scripts/setup-agents-wizard.sh:290	sort -V -k1,1	
F9	MED	scripts/test-setup-agents-wizard.sh:344, 609	stat -c %a	

ID	Sev	File : line	Literal	Why port
F10	HIGH	_tools/od/start-daemon.sh:6, 7	/Applications/Open Design.app/...	mac is no all. C daem and dow silen noth
F11	HIGH	_tools/translate-fleet.sh:56, _tools/ helixtranslate-container.sh:45, _tools/ distribute-helixtranslate.sh:66, 68, 73, 74	thinker.local, amber.local, milosvasic@amber.local	The two the p runt matc host any fleet host reso
F12	HIGH	_tools/helixtranslate-container/ run.sh:12, _tools/helixtranslate- local.sh:33	/usr/local/bin/ unified-translator	Writ side boun over or h else runs redi failu norm
F13	MED	_tests/playwright.config.js:30, 34, _tests/visual-effects.config.js:25	port: 8401, port: 8082	Fixe with cheo conc reus fals occu hard

ID	Sev	File : line	Literal	Why port
F14	MED	20 files under _tests/ — see below	http://localhost:8401, http:// localhost:8082, :8481, :8482, :8483	42 o no e risk anot alre 8401 that pass site. all- inte alre corr (pro \\ \\ mod
F15	LOW	10 files under _tools/gen/ and _tools/ — see below	"glm-4.5-flash", "llama-3.3-70b- versatile", "gpt- oss-120b", "zai- glm-4.7"	15 o bare cons argv or ra silen tran rath _too tran alre corr (TRAI {TRA mist late was mea wha rule

ID	Sev	File : line	Literal	Why port
				the g beca blan <fil also two (row
F16	MED	design-system/diagrams/_prompts/build-and-generate.sh:13	OD_DAEMON_URL=http://127.0.0.1:4321 BYOK_PROVIDER=openai BYOK_BASE_URL=https://api.mistral.ai/v1 BYOK_MODEL=codestral-latest	Expo :-, s wha set. gene righ awa The resta read go.m pack milo vers silen assu pack file i \$11.4 scri gate defin
F18	LOW	.github/workflows/ci.yml.disabled:166, 171, 178, 185, 189	go-version: '1.26', node-version: '20', ruby-version: '3.3', sudo apt-get ...	/bin exist bash This wrap tooli Tun ("a l com MAX or sl
F19	MED	upstreams/GitHub.sh:1	#!/bin/bash	
F20	LOW	_tools/watch-deploy.sh:30	sleep 1200	

ID	Sev	File : line	Literal	Why port
F22	LOW	_tools/watch-deploy.sh:19	cat /proc/uptime	idles cycle Linu echo fatal cont start siler blan The mod appl cal mod env conf so a capp char siler faili the r cata sam justi Cons to a addr over own AS D nam as th exar zen_ struc com the s defa /bin exist
F23	MED	11 files under submodules/LLMProvider/ pkg/providers/ — see §4.8	CerebrasModel = "llama3.1-8b", ZhipuModel = "glm-4- flash", model = "glm-4.5", ...	
F24	MED	submodules/LLMProvider/pkg/providers/ ollama/ollama.go:77, .../zen/ zen_http.go:48, 61, 74	http:// localhost:11434, http://localhost:4096	
F25	MED	21 files under submodules/LLMProvider/, submodules/RAG/, submodules/passage/,	#!/bin/bash	

ID	Sev	File : line	Literal	Why port
		submodules/verdict/ — see §4.8; plus 10 in submodules/containers/ that this row never listed		bash Sam four repo
F26	HIGH	submodules/{LLMProvider,RAG}/scripts/ host-power-management/install-host- suspend-guard.sh, .../challenges/scripts/ host_no_auto_suspend_challenge.sh	systemctl mask ..., journalctl --since ..., / etc/systemd/ sleep.conf.d/...	system be th man comm guar eithe chal ever null a no the g succ insta the p mish this catch
F27	MED	submodules/{LLMProvider,RAG}/ challenges/scripts/ host_no_auto_suspend_challenge.sh:80, 81	date -d "@\$(stat -c %Y ...)" -Iseconds \\ \\ stat -c %y ...	Both GNU \\ br stat GNU ther at al appe

ID	Sev	File : line	Literal	Why port
F28	LOW	submodules/RAG/challenges/scripts/rag_unit_challenge.sh:46, 54	GOMAXPROCS=2 nice -n 19 go test ... -p 1	A de cap, liter chal tune runs box get t
F29	HIGH	_tools/pdf/build-pdfs.sh:129 (at HEAD before 2026-09-01)	m=\$(stat -f %m "\$f" 2>/dev/null \\ stat -c %Y "\$f" 2>/dev/null \\ echo 0)	The prot com desc corr core syst argu pars oper %m (s by 2: SUC file, files STD The rc=1 spell epoc Mea

ID	Sev	File : line	Literal	Why port
				core byte epoch "\$ne faile <i>expr</i> ever first, pinn the I proc OLD mea the n selec siler still radi prec prob inop rend NOT dete Wea cont refer SOUR and no d no / writ runs iden mec. brok it gu held else. cons AUD rem

ID	Sev	File : line	Literal	Why port
----	-----	-------------	---------	-------------

F31 | LOW | workshop/pipeline/run_audit.sh:78, 104, 120, workshop/pipeline/run_whispercpp.sh:64, 87, workshop/platform/backend/cmd/bench/main.go:223, 229, .../pkg/entail/eval_test.go:325, 353, 386, 436, .../pkg/search/lumenvec_live_test.go:42 | /proc/loadavg | Linux-only. 12 occurrences recording load average as run provenance. The two Go readers degrade (err is returned, or the string becomes "unknown"), but the two shell readers do not: \$(cut -d' ' -f1-3 /proc/loadavg) writes *No such file or directory* to stderr and substitutes the empty string, so run_whispercpp.sh embeds "load_start": "" in the run manifest it publishes as evidence. **The coupling is wider than this gate can see:** the same two scripts also call nproc (GNU coreutils; BSD spells it sysctl -n hw.ncpu) and /usr/bin/time -f (GNU time; BSD time has no -f), neither of which is in any detector class here. The honest statement is that the transcription pipeline is Linux-only by construction, not that it has one Linux literal. | uptime is POSIX and prints the same three figures; or guard with [-r /proc/loadavg] and omit the key when it is absent, exactly as write_verification_sidecar already omits video_mtime_epoch on a host that cannot report one. **A fix lands in workshop's own upstream and returns as a gitlink bump** — it is not made from this tree. |

F32 | MED | workshop/platform/backend/cmd/bench/main.go:48, .../gates/bench-answers.sh:36, .../pkg/answer/ollama.go:109, .../pkg/search/lumenvec_live_test.go:175, workshop/platform/gates/prove-server-unity.sh:535 | http://127.0.0.1:8087, http://127.0.0.1:8091, http://127.0.0.1:11434 | **workshop has no settings seam, and the module is already inconsistent with itself about it.** cmd/workshop-server and cmd/workshop-ask route every default through a local env(key, def) helper *and* a flag, so WORKSHOP_HTTP_ADDR / WORKSHOP_OLLAMA_URL / -http all override — that is the seam, written twice, once per binary. These five sites do not use it: bench/main.go has a flag default and no env

layer; bench-answers.sh accepts only \$1; ollama.go and lumenvec_live_test.go write the literal as a bare in-package fallback; prove-server-unity.sh has no override at all, so the gate probes a fixed host:port and reports UNDETERMINED on any host that serves elsewhere. | **Yes, workshop needs its own seam — it must not reuse submodules/LLMProvider/pkg/settings.** That package is the seam for the F23/F24 class *inside LLMProvider*, keyed LLMPROVIDER_<PROVIDER>_*; workshop is a separate Go module (github.com/milos85vasic/workshop_curriculum/...) with its own WORKSHOP_* key space already established by the two env() helpers. Importing LLMProvider for a getenv wrapper would add a dependency to acquire the wrong prefix. The remediation is to promote the duplicated env/envInt/envBool helpers into one platform/backend/pkg/settings and route these five sites through it. **Lands in workshop's upstream, returns as a gitlink bump.** |

F33 | MED | workshop/pipeline/detect_media.sh:115, 162 | readlink -f -- "\$bin" 2>/dev/null || printf '%s' "\$bin" | readlink -f is a GNU extension. The || fallback keeps the script alive, so this is a **degradation, not a crash** — and what degrades is the one thing the probe exists to report. _portable.sh records the measured trap it was written for: ffmpeg on this host is a symlink into a Playwright browser cache whose target is ffmpeg-linux, so it answers -version and rejects -show_format. Naming the **target** is what makes the UNUSABLE verdict actionable; on BSD this reports the symlink and the operator learns nothing. Same defect class as F7. | workshop/scripts/_portable.sh is this module's home for exactly this and carries **no path-resolution primitive yet** — its functions are sha256_file, sha256_stream, have_sha256, file_size_bytes, file_mtime_epoch, iso8601_utc, probe_media, mp4_ftyp_brand, write_verification_sidecar, split_numeric. Add resolve_path there following that file's own stated rule — *validate the output, never trust the exit status* — trying readlink -f, then python3 -c os.path.realpath, then cd "\$(dirname x)" && pwd -P, and accepting a candidate only if it is a non-empty absolute path. Then detect_media.sh sources it instead of spelling this a third time.

Lands in workshop's upstream, returns as a gitlink bump. |

F34 | MED | scripts/verify-content-boundary.sh:998 | sed -i "s\|^\$(printf '%s' "\$p" \ | sed 's/[[][\./.^\$*]/\&/g')\t\$role\t|\$p\tunverified\t|" "\$FLEET" 2>/dev/null | GNU in-place form: BSD/macOS sed -i consumes the next argument as a backup suffix, so this either edits the wrong thing or errors. Same defect as F3 and F6, and it is **not** inside a submodule — it is the new umbrella-root script commit 695c22d3 added, which is why it appeared in the same run as the workshop/ findings. The 2>/dev/null makes it silent, and what it silently fails to do is **demote a fleet row to unverified** after the script has

already decided the repository could not be read — so on BSD the row stays private/public and is then scanned as if it were empty. | Write to a temp file and mv, which is the remediation F3 and F6 already carry. **NOT FIXED HERE:** scripts/verify-content-boundary.sh was explicitly out of bounds for the change that recorded this finding. Recorded so it is never silently omitted (§11.4.6). |

F14 file list (42 occurrences): _tests/tests/aria-footer-l10n-runtime.spec.js (2), download-switcher-perlang.spec.js (2), home-lang-nav.spec.js (3), interactive-behavior.spec.js (2), link-integrity.spec.js (2), milosvasic-ru-ally.spec.js (1), milosvasic-ru-features.spec.js (1), milosvasic-ru-il8n-articles.spec.js (1), milosvasic-ru.spec.js (1), perf-budget.spec.js (4), security-hardening.spec.js (4), seo-meta.spec.js (6), ui-l10n-chrome.spec.js (2), vasic-digital-ally.spec.js (1), vasic-digital-features.spec.js (1), vasic-digital-il8n-articles.spec.js (1), vasic-digital.spec.js (1), _tests/tools/motion-audit.cjs (5), _tests/ui-l10n2-verify.js (1), _tests/visual-effects.spec.js (1).

F15 file list. The 2026-08-31 snapshot said **15 occurrences:** _tools/gen/repair_ui_terms.py (1), translate-ui.py (1), translate_aria_footer.py (3), translate_home.py (3), translate_ui_all.py (1), translate_ui_batch.py (1), translate_ui_chunked.py (1), translate_ui_headroom.py (3), translate_ui_slow.py (1).

That count is superseded, not wrong-at-the-time: the gate measures 17, and the difference is fully accounted for rather than waved at. translate_home.py is 4, not 3 (its line 312 function docstring), and translate_ui_chunked.py is 2, not 1 (its line 2 module docstring). Both extras are the §4.6 rows 12–13 that the blanket <file> MODEL * baseline rows suppressed silently. The file list itself is otherwise exact, and it is the count — not the roster — that moved. **All 17 are CLOSED in source as of 2026-09-02**; the per-line enumeration is in the “F15 — CLOSED in source” block near the top of this document. Re-derive the roster with the rules stripped from a scratch copy of the gate, which is how the 17 were enumerated in the first place.

4.5 Findings that were fixed by a concurrent agent while this audit ran

These were live when the sweep started and are **verified fixed** in the working tree as of the final run. They are recorded because withdrawing a finding silently would be the same bluff as inventing one.

ID	Was	Now
F1	ENVFILE=/etc/sysconfig/ollama (RHEL/SUSE only)	detect_env_file() honours \$OLLAMA_ENV_FILE, then asks systemd for EnvironmentFiles, then probes /etc/sysconfig/, / etc/default/, /etc/ conf.d/ in turn.
F2	systemctl / journalctl called unguarded in scripts/ollama-vulkan- remediation.sh	detect_service_manager() dispatches over systemctl+ps -p 1, launchctl on Darwin, and rc-service, all behind command -v; OS="\$(uname -s)" at the top.
F4	GGML_VK_VISIBLE_DEVICES / Vulkan assumed as the only accelerator stack	GPU_DRIVER/GPU_VENDOR are env-overridable and otherwise read off the running kernel.
F5	systemctl / journalctl unguarded in scripts/ lumen-reindex.sh	Both are now behind command -v ... >/dev/ null 2>&1.

4.6 Deliberately classified as NOT a defect

19 occurrences. Each was opened and read; none configures anything machine-specific.

#	File : line	Hit	Why it is not a defect
1	_tools/pdf/build- pdfs.sh:129 → _tools/pdf/ build-pdfs.sh:144, 189	GNUBSD stat -c %Y ... then stat -f %m ..., each validated	This is already the portable form — BSD first, GNU fallback, safe default. Flagging it would punish the fix. It is the reference implementation cited in F9. That entry was FALSE and is withdrawn (2026-09-01) — see F29. The line it exempted was a broken

#	File : line	Hit	Why it is not a defect
			command whose failure was masked, and the loose <code>stat -f %m</code> pattern matched the broken form too, so this row was suppressing the defect it advertised as the fix. The file now runs each spelling separately and validates that the OUTPUT is a bare integer (<code>portable_mtime</code> , plus an independent readback inside <code>--prove-mtime</code>), and the allow rule now matches that validated dispatch specifically — seeding the old one-liner back makes the gate flag it.
2	<code>_tools/translate/glossary.json:216</code>	MODEL "codestral"	An entry in the non-translatable technology-term list , alongside "Playwright", "pandoc" and "Jekyll". It selects no model.
3–6	<code>scripts/lumen-index-doctor.sh:113, 146, 160, 162</code>	MODEL, ENDPOINT	The file defines its own <code>env(name, default)</code> wrapper over <code>os.environ</code> (line ~86). Every literal is the last term of an <code>env(X)</code> or <code>cfg.get(Y)</code> or " <code><literal></code> " chain whose precedence is copied from lumen's own <code>applyEnvOverrides</code> . Line 146 is a docstring.
7–10	<code>scripts/test-setup-agents-wizard.sh:247, 290, 879, 1145</code>	OSPATH, GNUBSD, ENDPOINT	All four are test fixtures . 247 is a <code>grep -cE</code> pattern asserting the

#	File : line	Hit	Why it is not a defect
			wizard <i>never</i> writes / etc/sysconfig/ollama; 290 is an assertion title string quoting sort -V; 879 writes a synthetic throwaway repo to prove the sibling paths audit does <i>not</i> flag /etc; 1145 sets OLLAMA_HOST="http:// 127.0.0.1:1" because port 1 must be unreachable for the negative test to mean anything. A log line. The port is \${OD_PORT:-4321} from the line above, and 127.0.0.1 is the loopback constant, chosen deliberately so the daemon is not reachable off-box. Module docstring prose. MOOT 2026-09-02 — the judgement stands, but the occurrence is gone: the docstring now names UI_TRANSLATOR_PROVIDER / UI_TRANSLATOR_MODEL and states the defaults on a following line that carries no quote character, so the class no longer matches. It was reworded rather than converted into a # REASON: row, because an allow row would have bought a smaller number at the cost of
11	_tools/od/start-daemon.sh:10	ENDPOINT	
12	_tools/gen/translate_ui_chunked.py:2	MODEL	

#	File : line	Hit	Why it is not a defect
			hiding the line from the next reader. Function docstring prose. MOOT 2026-09-02 — same treatment: the docstring now says “the configured translator model (HOME_TRANSLATOR_MODEL, default gpt-oss-120b)”, with the literal on a line that carries no quote character. A provider registry: each row is (the vendor’s published API endpoint, that vendor’s API-key env var, that vendor’s own default model). The model is overridden by the documented --model flag. Nothing is bound to a machine. Path corrected 2026-09-01 late: the gate reports this at the <i>gitlink</i>, not at the vendored spec-kit copy. submodules/superspec is third-party upstream (WangX0111/superspec), explicitly outside the owned-submodule set, so it is reported as a NOTE and excluded from the verdict per §11.4.29 — reported so it is never silently omitted, never counted as this tree’s failure. Upstream’s pin is
13	_tools/gen/ translate_home.py:312	MODEL	
14–18	_tools/ review_translation.py:64–68	MODEL	
19	specify/extensions/ superspec/.github/ workflows/ci.yml:21 → submodules/ superspec/.github/ workflows/ci.yml:21	TOOLVER python- version: "3.12"	

#	File : line	Hit	Why it is not a defect
			upstream's to change. It is the only third-party NOTE in the current run.

Two further observations recorded as INFO, below the defect bar:

- `_tools/od/od-mcp-call.mjs:10` — `process.env.OD_MCP_SERVER || '/opt/homebrew/lib/node_modules/...'`. Env-overridable, therefore not a defect by the rule in §3. Worth noting anyway that the **default** is macOS/Homebrew-only, so on Linux the variable is not optional in practice.
- `.github/workflows/ci.yml.disabled:119,121` — `branches: [main]`. A workflow trigger naming its own default branch is repository configuration, not a machine assumption. The GITREF class deliberately does not match `YAML branches: keys`.

4.7 Known blind spots

Stated rather than left implicit:

- The gate reads `git ls-files`, so **untracked files are invisible to it**. At the time of writing `scripts/continuation-check.sh`, `scripts/ollama-tune.sh` and `scripts/verify-provider-ci.sh` exist in the working tree but are untracked, and were therefore not scanned. They will be scanned the moment they are added.
- ~~Submodule interiors (`milosvasic.ru/`, `vasic.digital/`, `design-toolkit/`, `ai-interviewing/`, `monetization/`, `submodules/`) are gitlinks and are out of scope. Each needs its own copy of this gate.~~ **CLOSED 2026-09-01**. This was not a scope choice, it was a defect: `git ls-files` reports a gitlink as one directory entry, and the gate's `[ -f ... ]` filter dropped it, so `--list` reported 179 files with **zero** inside any declared submodule. The fleet is now derived from `.gitmodules`, ownership from `helix-deps.yaml`, and the sweep covers 1 762 files across 14 repositories. The gate refuses to report a verdict if a swept submodule contributes zero files. See §4.8.
- The env-override test is **per line**. A literal assigned on one line and guarded three lines later is reported. Two such cases were found and are handled by named allow rules rather than by loosening the rule.
- `MODEL` only fires on a **quoted** model id, so a model name written into a docstring is ignored — but so would one built by string concatenation.

- The `isfallback` escape hatch is **single-line**. A composite literal whose field name declares it a fallback — `FallbackModels:`
`[]string{` — puts its entries three lines below the name, where the heuristic cannot see them. Measured 2026-09-01: this accounts for 107 of the 476 fleet findings. The detector was deliberately **not** widened to compensate; see §4.8.
-

4.8 2026-09-01 — the fleet expansion, and its triage

`submodules/LLMProvider` and `submodules/RAG` were adopted as root submodules on 2026-09-01 under §11.4.74 (reuse before reimplement). Adding the two gitlinks put 212 more files into the scan universe. **476** findings appeared, all of them inside those two repositories and none anywhere else in the tree.

They are **newly visible, not newly created**. They are also not a reason to narrow the scan: this gate was blind to all nine submodules until that blindness was fixed the same day, and that blindness is why a `/Users/...` literal survived in `ai_interviewing`. A blind instrument reports PASS.

Reproduced inventory (`bash scripts/audit-environment-assumptions.sh`, 2026-09-01, 1 762 files / 14 repositories):

Class	Count	Where
MODEL	404	270 in <code>*_test.go</code> , 134 non-test
SERVICE	26	host-power-management + challenge scripts
ENDPOINT	22	18 in <code>*_test.go</code> , 4 non-test
SHEBANG	17	<code>#!/bin/bash</code>
GNUBSD	4	<code>date -d / stat -c</code>
PARALLEL	2	<code>GOMAXPROCS=2</code>
HOSTNAME	1	<code>http://invalid.local</code> in a test
Total	476	

The triage rule

The SCOPE of an allow row must equal the SCOPE of the reason justifying it. A reason that is structural for a whole file earns a file+class row. A reason that is about one literal earns a row pinned to that literal. Anything that is neither is DEBT: a BASELINE row carrying a finding id from this document.

CLASS is never widened to * anywhere in this section, so a new finding of a *different* class in an already-covered file still fails the gate.

JUSTIFIED — 412 occurrences, 153 rows.

- **(A) *_test.go — 289 occurrences, 34 file+class rows.** go build excludes these files from every production binary, so no literal in them can configure a deployment; the literal is the value the assertion compares against, and deriving it from the environment would delete the assertion. Verified: 28 of the 30 test files carrying findings drive `httptest.NewServer`; the other two use a deliberately unreachable port (`127.0.0.1:1`) or assert a constructor default, and their one live path is capability-guarded and `t.Skip()`s. The reason is structural for the whole file, so the row is file-scoped — but still class-scoped, which probe P7 proves.
- **(B) FallbackModels / SupportedModels entries — 107 occurrences, 107 literal-pinned rows.** The last-resort catalogue used **only** when the live ModelsEndpoint discovery call fails. Measured: **all 107** non-test MODEL list elements sit inside `FallbackModels: []string{ (106)` or `SupportedModels: []string{ (1); not one` is a default selection. These are remote-vendor API ids, bound to no machine, OS or deployment. This is exactly the shape the gate's own `isfallback` heuristic exempts — the heuristic just cannot see a field name three lines up. **The detector was deliberately not widened to reach rc=0**; widening a detector to clear a finding is the failure mode this section exists to avoid.
- **(C) zen.go model-id constants — 5 occurrences, 5 pinned rows.** A named catalogue with deprecation notes, for callers to pass in. The only default, `DefaultZenModel`, is a symbol reference and never fires.
- **(D) codestral pattern false positives — 11 occurrences, 7 pinned rows.** A provider id (`ProviderID: "codestral"`), a Go variable name (`codestralResp.Usage.PromptTokens`), an i18n message key, and that message's English text. None is a model id.

DEBT — was 64 occurrences, F23–F28, 37 rows. RE-MEASURED
2026-09-01 late: all six rows are now at 0. The previous revision of this table left F25 at 4 and F27 at 2; both were fixed after it was written, and both are corrected below rather than quietly reduced.

F23–F28 being at 0 is not the same as this document being green, and the distinction is the whole point of a baseline. The gate still prints **708 baselined occurrences across 228 files** on every clean run (see the re-derived block at the top). Those are other findings, in other classes, most of them in `*_test.go` and vendor-pinned catalogues triaged in §4.8. Closing F23–F28 closed the rows that had finding ids; it did not empty the ledger.

The `Now` column is a re-measurement, not a restatement. Re-derive any row with the command beside it; do not quote these numbers as current without re-running.

Finding	Was	Now	Files / evidence
F23 default model on a production path	11	0 in the 11 named files, +4 newly-recorded in zen/zen.go, now also 0	The 11 files were converted to <code>settings.Model("<provider>", <Const>).zen/zen.go was never in this row and carried the SAME defect four more times (:409 endpoint, :413 + :444/:449 model, :433 timeout) — found by checking every provider for a SECOND occurrence after zen_http.go turned out to carry it twice. Now routed through pkg/settings. Re-derive: <code>grep -rn 'settings\.Model(' submodules/LLMProvider/pkg/providers/ollama.go and zen_http.go were converted. zen/zen.go:409 froze ZenAPIURL, and :451 was worse: NewZenProviderAnonymous passed ZenAPIURL positionally to NewZenProviderWithRetry, bypassing that constructor's own empty-check, so the override worked on every path except that one. Both fixed; GetCapabilities now reports p.baseURL instead of</code></code>
F24 frozen default endpoint	4	0, +2 newly-recorded in zen/zen.go, now also 0	

Finding	Was	Now	Files / evidence
F25 <code>#!/bin/bash</code>	17 (+4, see below)	0 — CLOSED	the constant it used to report regardless of the endpoint in force. LLMProvider 0, RAG 0, containers 0, passage 0, verdict 0, all five re-measured 2026-09-01 late. The 4 that stood in this cell, and the sentence “submodules/passage 2 and submodules/verdict 2 REMAIN — F25 is NOT closed”, are WITHDRAWN: both were true when written and were fixed afterwards in each submodule’s own checkout. The 10 occurrences in submodules/containers this table never carried a row for (recorded only in <code>.environment-assumptions-allow</code>, whose header admits the missing finding ids) are fixed as well, and their 12 allow rows have been deleted — see the paragraph under this table. Re-derive per repo, with the path form that actually reads the file: <code>git -C <repo> ls-files \ while read -r f; do head -1 "<repo>/\$f" \ grep -q '^#!/bin/bash' && echo "\$f"; done \ wc -l</code>
F26 unguarded <code>systemd</code>	26	0 in LLMProvider and RAG	All four named files now carry a real command <code>-v systemctl / command -v journalctl</code> capability guard, so the gate’s file-scope guard clears the class. Re-derive: <code>grep -cE 'command -v (systemctl\ journalctl)' <file> — measured 1, 2, 1, 2.</code>
F27 GNU-only <code>date/stat</code>	4	0 — CLOSED	Fixed with the validating <code>portable_mtime</code> in LLMProvider, RAG and containers; see the F27 row above. “submodules/containers/...:80–81 still carries it and is NOT fixed” is WITHDRAWN: re-measured 2026-09-01 late, that

Finding	Was	Now	Files / evidence
F28 frozen concurrency	2	0	file carries portable_mtime at :164 with the bare-integer validation at :166, the date dispatch at :176, and assert_undet at :197/:200. Re-derive: <code>grep -n 'portable_mtime\ assert_undet' submodules/containers/challenges/scripts/host_no_auto_suspend_challenge.sh</code> RAG/challenges/scripts/rag_unit_challenge.sh now reads <code>GOMAXPROCS="\${GOMAXPROCS:-2}"</code> (:51) and <code>-p "\${GOTEST_P}"</code> (:58, :66) — exactly the remediation this row prescribed.

Allow-list rows for closed items have been deleted, and the count “24 rows” is CORRECTED to 12. Measured 2026-09-01 late against HEAD:

```
git diff HEAD --numstat -- .environment-assumptions-allow # 812
added / 36 removed (all lines)
git diff HEAD -- .environment-assumptions-allow | grep -cE '^-[^\-#]' # 12 rule lines removed
git diff HEAD -- .environment-assumptions-allow | grep -cE '^+[^+#]' # 168 rule lines added
```

All **12** removed rules are submodules/containers/... SHEBANG/SERVICE rows — exactly the F25/F26 items closed in that repository. They were dead: the defect each named was gone, so the rule suppressed nothing while leaving a standing exemption at that path. That rot stayed invisible until `scripts/audit-environment-assumptions.sh` grew a stale-rule census; see “Allow-list rot” in that script’s header. A stale BASELINE is not untidiness — it is an assertion that a defect still exists, and it was false.

The 168 additions are recorded, not glossed. A claim circulated that the audit went red→green “without a single allow-list addition, 4 rows removed”. It is **not reproducible from this tree** and is not adopted: the only window this document can observe is working-tree-vs-HEAD, and over that window the file gained 168 rule lines and lost 12. The claim may describe a narrower interval between two uncommitted states, which nothing here can see. Recording the measurable window and saying so is the honest form; asserting a figure that cannot be re-derived is not (§11.4.6). **Consequence, stated plainly: this document**

does NOT claim the green was achieved by fixes alone. Whoever adds an allow rule owes it a finding id, and 168 new rows are 168 assertions that somebody should be able to re-derive.

HONEST BOUNDARY (§11.4.6). “FIXED — 0, deliberately”, which stood here at one point, was **withdrawn** and is not reinstated by the fact that the number is now 0 again. Its reasoning — that §11.4.29 forbids editing a consumed repository from here — remains correct, and none of the fixes above were made from this tree: each landed in the submodule’s **own** checkout and returns as a gitlink bump. What was wrong was treating “we may not fix it here” as evidence that there was nothing to fix. The number is 0 today because the work was done upstream, which is a different claim and re-derivable with the commands beside each row.

F30 — a frozen default DELIBERATELY LEFT, recorded so it is never mistaken for an oversight. submodules/LLMProvider/pkg/providers/junie/junie_cli_stub.go freezes the model id "junie-1" at 5 sites, none routed through pkg/settings:

	line	site
	72	var knownJunieModels = []string{"junie-1"} return
	113	JunieCLIConfig{Model: "junie-1", MaxTokens: 4096} return
	118	JunieACPCConfig{Model: "junie-1", MaxTokens: 4096} func (p *JunieCLIProvider)
	122	GetCurrentModel() string { return "junie-1" }
		func (p *JunieACPPProvider)
	125	GetCurrentModel() string { return "junie-1" }

It is a **stub for a capability the standalone module does not have**, so there is **no live path on which the value could be unfrozen** — an env layer over a constant that no request ever reaches would be

adaptability theatre, and it would be indistinguishable, in this document, from the real F23 fixes. It is therefore left as it is, deliberately, and written down here instead.

Two further facts about it, so nobody re-derives a contradiction: the gate's MODEL class does **not** fire on the literal junie-1 (the file is not among the 228 baselined files), and there is **no allow rule mentioning junie** — `grep -n junie .environment-assumptions-allow scripts/audit-environment-assumptions.sh` returns nothing. So F30 is carried by this document alone. If the stub ever acquires a live path, F30 becomes an ordinary F23 and must be fixed, not re-justified.

Four more rows, added the same hour and not part of the 476. `submodules/passage/upstreams/vasic_digital_{github,gitlab}.sh` and `submodules/verdict/upstreams/vasic_digital_{github,gitlab}.sh` were reported as THIRD-PARTY notes at 11:20 and were **gating** findings by 11:40, without either file changing: a concurrent agent added both repositories to `helix-deps.yaml` `deps[]` at 11:36, and ownership is derived from that file. They are the same one-line defect as F19/F25 and are recorded under F25 rather than left to hold the gate red for a reason nobody triaged.

§1.1 narrowness proof — 10 of 10 caught

An allow row that swallows a real defect is worse than the finding it hides, so every row family above was probed: a genuinely unportable construct was seeded into a file the new rows **cover**, and the gate had to still catch it. Two dimensions — **CLASS** (a file+class row must not swallow a different class in that file) and **LITERAL** (a pinned row must not swallow a different literal of the same class in that file). Run 2026-09-01; every file restored byte-identically, verified by `sha256sum` and by `git status --porcelain` returning empty in both submodules; the gate returned to `rc=0` afterwards.

Probe	Dim	Seeded into	Seed	Expected
P1	LITERAL	cerebras/cerebras.go (F23 row pins = "llama3.1-8b")	var ... = "llama-3.1-405b-instruct"	MODEL
P2	LITERAL	ollama/ollama.go (F24 row pins http://localhost:11434)	var ... = "http://localhost:9999"	ENDPOINT
P3	CLASS	upstreams/github.sh (F25 row is SHEBANG *)	sed -i "s/a/b/"	GNUBSD

Probe	Dim	Seeded into	Seed	Expected
P4	CLASS	install-host-suspend-guard.sh (F26 row is SERVICE *)	readlink -f	GNUBSD
P5	CLASS	host_no_auto_suspend_challenge.sh (F27 row is GNUBSD *)	echo "seeded.local"	HOSTNAME
P6	CLASS	rag_unit_challenge.sh (F28 row is PARALLEL *)	readlink -f	GNUBSD
P7	CLASS	cerebras/cerebras_test.go (row (A) is MODEL *)	var ... = "/etc/ sysconfig/ ollama"	OSPATH
P8	LITERAL	cloudflare/cloudflare.go (15 pinned (B) rows)	var ... = "llama-3.1-405b- turbo"	MODEL
P9	LITERAL	zen/zen.go (5 pinned (C) rows)	var ... = "glm-4.9-frozen"	MODEL
P10	LITERAL	codestral/codestral.go (7 pinned (D) rows)	var ... = "codestral-2501"	MODEL

P8 is the load-bearing one: cloudflare.go carries 15 justified MODEL rows and still fails on a 16th literal, which is what distinguishes a pinned row set from a cloudflare.go MODEL * blanket.

4.9 2026-09-01 — the workshop gitlink bump, and its triage

Commit 695c22d3 moved the workshop gitlink from a near-empty commit to one carrying **247 paths**. (It was cdb5351 when this finding was recorded, 25fe585e after the 2026-09-01 authorized content-boundary rewrite, and 695c22d3 after the second one on 2026-09-02; only the last resolves. Mappings: [docs/content-boundary-incident-2026-09-01.md](#) §8B and §11.4.) Both this gate and scripts/audit-hardcoded-paths.sh scan submodule interiors, so the findings became visible in a single index update. **Newly visible, not newly created** — the identical pattern to §4.8, and §4.8's triage rule is the one applied here.

Reproduced inventory

bash scripts/audit-environment-assumptions.sh — **rc=1, 82 findings across 22 files**, 1 982 files / 14 repositories:

Class	Count	Where
ENDPOINT	48	29 in captured JSON fixtures, 12 in *_test.go, 7 non-test
MODEL	14	4 in captured JSON fixtures, 8 in *_test.go, 2 non-test
OSPATH	12	/proc/loadavg, in 5 files
GPU	3	accelerator refusal in run_whispercpp.sh
GNUBSD	3	2 × readlink -f, 1 × sed -i
TOOLVER	1	inside a captured JSON fixture
SERVICE	1	inside an ASR transcript
Total	82	81 inside workshop/, 1 at the umbrella root

bash scripts/audit-hardcoded-paths.sh — **rc=1, 8 occurrences across 3 files**, 4 824 files / 14 repositories. All three are evidence files under workshop/.

A count of 77 (48 ENDPOINT / 14 MODEL / 12 OSPATH / 2 GNUBSD / 1 SERVICE) circulated for this bump and is CORRECTED to 82. It omitted the 3 GPU and 1 TOOLVER findings entirely, and counted 2 GNUBSD where the run reports 3 — the third being scripts/verify-content-boundary.sh:998 (F34), which is at the umbrella root rather than inside workshop/. Re-derive; do not quote either figure without running the gate.

Dispositions — 62 JUSTIFIED, 20 DEBT, 0 FIXED

Measured by the gate itself, not by hand: justified occurrences moved **464 → 526 (+62)** and baselined occurrences moved **708 → 728 (+20)** across the change, and 62 + 20 = 82.

JUSTIFIED — 62 occurrences, 26 rows. No row uses CLASS *, and no row is file-scoped where a literal would do.

- **(A) Captured and generated artifacts — 35 occurrences, 6 rows.** transcript.segments.json is ASR output: segment 267's text field is a speaker *saying* that ollama can be restarted with systemctl. The two pkg/entail/fixtures/captured-claims*.json files are written by capture_test.go via ENTAIL_CAPTURE_OUT from a live answering run, and every flagged string sits inside "content": — documentation passages the model was given as context. Pinned to "text": " and "content": ", so a real endpoint or service key added to either file still fails.
- **(B) *_test.go — 12 occurrences, 8 rows.** Precedent (A) of §4.8, but pinned tighter than file+class in every case. net.Listen("tcp", "127.0.0.1:0") is an OS-assigned ephemeral port — the opposite of a frozen one. 127.0.0.1:1 is deliberately closed and localhost.invalid is RFC-2606-reserved and guaranteed never to resolve; provider_test.go exists to prove locality is **resolved and not substring-matched**, so those literals *are* the assertions.
- **(C) The literal is the last term of an env-override chain — 11 occurrences, 8 rows.** Identical in shape to the scripts/lumen-index-doctor.sh rows in the embedded ALLOW_RULES.cmd/workshop-{server,ask} wrap every default in env(KEY, "<literal>") *and* expose it as a flag; pkg/entail uses envOr(KEY, "<literal>") (defined at capture_test.go:200). Two independent overrides is the most configurable form available. Rows are pinned to env("WORKSHOP_ / envOr("WORKSHOP_, so a **bare** literal in the same file is still a finding — which probe W4 proves.
- **(D) A measurement record naming what it measured — 1 occurrence, 1 row.** pkg/answer/http.go:475 names qwen2.5:3b-instruct-q4_K_M inside the provenance sentence /api/ask/status returns beside its latency figures. Substituting the model in force would attach numbers measured on one model to a different one — falsification, not adaptability.
- **(E) GPU refusal is not a GPU assumption — 3 occurrences, 3 rows.** GGML_VK_VISIBLE_DEVICES=-1 and CUDA_VISIBLE_DEVICES=**disable** every accelerator and are a no-op on a host with neither runtime; the whisper.cpp build they drive records itself as "*source, CPU-only, all GPU backends compiled OFF*". Removing them would make the run depend on whatever accelerator happened to be present, which is the defect this class catches. The third is the run manifest recording the refusal it just made.

DEBT — 20 occurrences, 12 rows, F31–F34. Real, known, unfixed; every row BASELINE with a finding id. F31 (12), F32 (5), F33 (2), F34 (1).

FIXED — 0, and that is a decision, not an omission. 81 of the 82 findings are inside workshop/, which is a **submodule**. The precedent in .hardcoded-paths-allow's baselined block governs: a fix lands in the submodule's own upstream and returns as a gitlink bump. The change that performed this triage was explicitly forbidden from committing, from bumping any gitlink, and from editing scripts/verify-content-boundary.sh — so a fix for F31–F34 could not have been *delivered* from here even where the remediation is known and cheap. Each of F31–F34 therefore carries its remediation in full, in the file that will land it, rather than a fix stranded in a working tree.

§1.1 narrowness proof — 12 of 12 caught

Same two dimensions as §4.8. Seeded into the live tree 2026-09-01, gate re-run, then every file restored from a pre-seed copy and the restore **proved** with sha256sum -c (10 of 10 OK) plus git -C workshop status --porcelain returning empty. Both gates returned to rc=0 afterwards.

Probe	Dim	Seeded into	Seed	Expected
W1	CLASS	transcript.segments.json ((A) row is SERVICE "text": ")	"restart_cmd": "systemctl restart ollama",	SERVICE
W2	LITERAL	captured-claims.json ((A) rows pin "content": ")	"endpoint": "http://127.0.0.1:8087",	ENDPOINT
W3	LITERAL	descriptor_test.go ((B), 4 pinned rows)	var seedModel = "qwen2.5:3b-instruct-q4_K_M"	MODEL
W3b	LITERAL	descriptor_test.go (same 4 rows)	var seedEndpoint2 = "http://127.0.0.1:8087"	ENDPOINT
W4	LITERAL	cmd/workshop-server/main.go ((C) rows pin env("WORKSHOP_)	var seedFrozen = "http://127.0.0.1:8087"	ENDPOINT
W5	LITERAL	cmd/workshop-server/main.go (same)	var seedEmbed = "jina-embeddings-code-cpu"	MODEL
W6	LITERAL	pkg/answer/http.go ((D) row pins the provenance sentence)	var seedProvModel = "qwen2.5:3b-instruct-q4_K_M"	MODEL
W7	LITERAL			GPU

Probe	Dim	Seeded into	Seed	Expected
		run_whispercpp.sh ((E) row pins ...=-1)	export GGML_VK_VISIBLE_DEVICES=0	
W8	LITERAL	run_audit.sh (F31 row pins /proc/loadavg)	cat /etc/sysconfig/ollama	OSPATH
W9	LITERAL	pkg/answer/ollama.go (F32 row pins ...:11434)	var seedAlt = "http://127.0.0.1:9999"	ENDPOINT
W10	LITERAL	detect_media.sh (F33 row pins the whole \\ dispatch)	readlink -f /tmp/seedprobe	GNUBSD
W11	FILE	evidence/answering/entailment-2026-09-01.txt — a sibling of the three allow-listed evidence files, in the same directory	SEEDED /run/media/.../workshop/seed-probe	hardcode paths

W4 and W5 are the load-bearing pair: cmd/workshop-server/main.go has **17** lines matching env("WORKSHOP_ and still fails on a bare literal written beside them, which is what distinguishes an env-chain row from a main.go ENDPOINT * blanket. W11 is the equivalent for the file-scoped hardcoded-paths list: three named evidence files are exempt and a fourth in the same directory is not.

Two seeds initially reported as MISSES were investigated rather than rewritten, and the cause is a detector limit, not an allow-list leak. "endpoint": "http://10.0.0.5:8087" and "http://192.168.1.50:8087" were not flagged at all. Neither line contains any allow row's MATCH, so no row could have swallowed them; the ENDPOINT class is emitted only for localhost, 127.0.0.1, 0.0.0.0 or [Pp]ort[]*[:=][]*NNNN (scripts/audit-environment-assumptions.sh:1118–1121). **RFC-1918 and other routable host:port literals are outside this gate's ENDPOINT coverage entirely.** That is recorded here as a known blind spot; the probes were re-run with loopback spellings, which the detector does see, and both were then caught (W2, W3b). The detector was **not** widened to make the probe pass — widening a detector to change a result is the failure mode §4.8 exists to avoid.

Allow-list rot

bash scripts/audit-environment-assumptions.sh --stale-rules: **2 stale of 401 before, 2 stale of 439 after**. None of the 40 rows added here is stale, and neither stale row is one of them — both predate this change and both live in the **embedded** ALLOW_RULES inside scripts/audit-environment-assumptions.sh (line 10 scripts/audit-hardcoded-paths.sh * *, line 134 BASELINE _tools/gen/review_ui_all.py MODEL *), a file the change that recorded this was not permitted to edit. They are reported, not cleared.

SUPERSEDED 2026-09-02 — both are now cleared. They were deleted, with each staleness claim re-derived by hand first; see “Re-derived state — 2026-09-02” at the top of this document for the per-rule evidence. --stale-rules now prints **0 STALE / 0 PATH-ABSENT of 436** and exits 0. The paragraph above is kept because the reason it existed — a rule that only ever grows is not a record of judgements — is the lesson, and because withdrawing it silently would be the same bluff as inventing it.

5. The gate

```
bash scripts/audit-environment-assumptions.sh          #
    audit this repo
bash scripts/audit-environment-assumptions.sh /path/to/repo #
    another checkout
bash scripts/audit-environment-assumptions.sh --list      #
    scanned universe
bash scripts/audit-environment-assumptions.sh --classes   #
    class reference
bash scripts/audit-environment-assumptions.sh --allow-list #
    effective rules
```

Exit codes — the project’s three-valued convention, same as scripts/lumen-index-doctor.sh:

rc	Meaning
0	clean — no <i>new</i> frozen assumptions
1	findings
	could not do its job — never collapse this into 1. A
2	broken checker is not a

rc	Meaning
	violating tree, and it is certainly not a clean one.

rc=2 is raised when: the target does not exist; the target is not a git working tree; a required tool (git, awk, grep) is missing; the scan universe is **empty** (an anti-bluff guard — a gate that finds nothing because it looked at nothing is the empty-build success this project has already been burned by); the allow-list is malformed; or the scan itself errors.

Runtime on this tree: **0.44–0.46 s** over 174 files, one awk process, no per-file subshells. That is inside a pre-push budget. To wire it in, add it alongside the existing gates in `scripts/pre-push-gates.sh` — deliberately not done as part of this audit, which was scoped to audit-and-gate only.

5.1 Allow-list

Embedded in `ALLOW_RULES` inside the script (this audit was permitted to add exactly two files to the tree, so the list could not be a third). An external `.environment-assumptions-allow`, or `$ENV_ASSUMPTIONS_ALLOW`, is read *in addition* when present, so it can be externalised later without touching the script.

```
# REASON: <why this literal is genuinely justified>
path/to/file          CLASS    substring-of-the-offending-line
```

```
# BASELINE: <known defect, AUDIT.md finding id>
path/to/file          CLASS    substring-of-the-offending-line
```

`PATH` may end in `*` to prefix-match; `CLASS` and the substring may each be `*`.

Every rule **must** carry a `# REASON:` or `# BASELINE:` in the contiguous comment block directly above it. A rule without one is a malformed allow-list and exits 2 — an unexplained suppression is indistinguishable from a bluff.

The two kinds are not interchangeable:

- **REASON** — genuinely justified forever. Counted, otherwise invisible.
- **BASELINE** — a **real, known, unfixed defect**, deliberately carried so the gate can catch *new* ones. Baselines are counted and their files printed loudly on every clean run, so the tree can never go quietly

green over known breakage. A baseline is a debt, not a justification, and must cite a finding id from §4.4.

~~Current state: 17 REASON occurrences, 87 BASELINE occurrences. Those figures are from the 2026-08-31 umbrella-only population and are superseded.~~ They described 174 scanned files; the sweep now covers 1 802 files across 14 repositories, so they are not comparable and must not be quoted as current.

Re-measured 2026-09-01 late — `bash scripts/audit-environment-assumptions.sh` (**superseded 2026-09-02; kept as the dated observation it was**):

justified (allow-listed) occurrences	464
baselined occurrences	708 , across 228 files
allow rules, total	401 — 353 external + 48 embedded
stale allow rules	2

Re-measured 2026-09-02 — same command:

justified (allow-listed) occurrences	533
baselined occurrences	683 , across 216 files
allow rules, total	436 — 396 external + 40 embedded
stale allow rules	0

The old note behind the “87 vs 85” gap still holds and is kept because it explains a deliberate imprecision that survives into the current numbers: two file-wide MODEL baselines (`translate_ui_chunked.py`, `translate_home.py`) also absorb the two docstring false positives 12 and 13 in §4.6. That is a deliberate loss of precision in favour of a shorter list, recorded rather than smoothed over.

Re-derive rather than quoting any of the above; every one of these numbers moved within a single day.

6. Mutation proof

Per §1.1 of the constitution, every gate carries a paired mutation proving it catches regressions. All probes ran against **throwaway git repositories** and a **path-faithful mirror** of the scan universe, never against the live tree — other agents were editing this repository concurrently.

6.1 Synthetic probes — 32 of 32 pass

Group	Probe	Expected	Observed
A	clean throwaway tree	rc=0	rc=0 PASS
B	plant ENDPOINT const BASE = 'http://localhost:8401';	rc=1 + names file & class	PASS
B	plant PARALLEL pool = Pool(max_workers=12)	rc=1 + named	PASS
B	plant MODEL MODEL = "llama-3.3-70b-versatile"	rc=1 + named	PASS
B	plant OSPATH ENVFILE=/etc/ sysconfig/ollama	rc=1 + named	PASS
B	plant SERVICE systemctl restart ollama	rc=1 + named	PASS
B	plant PKGMGR sudo apt-get install -y jq	rc=1 + named	PASS
B	plant GNUBSD sed -i "s/a/b/" f.txt	rc=1 + named	PASS
B	plant HOSTNAME ssh milosvasic@thinker.local uptime	rc=1 + named	PASS
B	plant TOOLVER go-version: '1.26'	rc=1 + named	PASS
B	plant GITREF git push origin main	rc=1 + named	PASS
B	plant GPU nvidia-smi --query- gpu=name --format=csv	rc=1 + named	PASS
B	plant SHEBANG #!/bin/bash	rc=1 + named	PASS
C		rc=0	PASS

Group	Probe	Expected	Observed
	HOST="\${OLLAMA_HOST:-http://localhost:11434}"		
C	process.env.VD_BASE \\ \\ 'http://localhost:8401'	rc=0	PASS
C	int(os.environ.get("UI_WORKERS", "5"))	rc=0	PASS
C	comment-only line mentioning systemctl restart ollama	rc=0	PASS
C	command -v systemctl >/dev/null 2>&1 && systemctl restart ollama	rc=0	PASS
C	case "\$(uname -s)" in Linux) sudo apt-get install -y jq;; esac	rc=0	PASS
C	#!/bin/sh (POSIX, guaranteed location)	rc=0	PASS
C	for f in /etc/sysconfig/x /etc/ default/x /etc/conf.d/x	rc=0	PASS
C	fallback = "ordis/jina- embeddings-v2-base-code"	rc=0	PASS
D1	real allow rule suppresses the validated two-spelling dispatch in _tools/pdf/build-pdfs.sh (lines 144, 189)	rc=0	PASS
D2	add readlink -f to that same allow-listed file	rc=1, names readlink -f	PASS
D3	same allow-listed line in a different path	rc=1, names the other file	PASS
D5	a different, BROKEN stat line in the SAME allow-listed file — seed stat -f %m "\$1" \\ \\ stat -c %Y "\$1" \\ \\ echo 0 into portable_mtime	rc=1, names _tools/ pdf/ build- pdfs.sh line 144	PASS (added 2026-09-01)
D4			PASS

Group	Probe	Expected	Observed
	add systemctl restart ollama to the allow-listed file (rule is GNUBSD-scoped)	rc=1, names SERVICE	
E1	target directory does not exist	rc=2	PASS
E2	target is not a git working tree	rc=2	PASS
E3	empty scan universe	rc=2, not 0	PASS
E4	external allow rule with no # REASON:	rc=2	PASS
E5	the same rule with a # REASON:	rc=0	PASS
E6	git/awk/grep absent from PATH	rc=2	PASS

D2/D3/D4 together are the precision proof the allow-list needs: a rule suppresses exactly its own (path, class, literal) triple and nothing adjacent.

6.2 Planted into the real project files

Same 12 classes, planted one at a time into a **path-faithful mirror** of all 174 scanned files (identical baseline: 174 scanned, 87 baselined, 17 allowed, rc=0), then reverted:

Class	Host file	Result
ENDPOINT	_tools/gen/build.sh	rc=1, offender named
PARALLEL	_tools/gen/data.go	rc=1, offender named
MODEL	_tools/gen/i18n.go	rc=1, offender named
OSPATH	_tools/render-articles.sh	rc=1, offender named
SERVICE	_tools/audit-hardcoding.sh	rc=1, offender named
PKGMRGR	_tools/translate-all-langs.sh	rc=1, offender named
GNUBSD	_tools/portfolio/self-validate.sh	rc=1, offender named
HOSTNAME	_tools/review-translations.sh	rc=1, offender named
TOOLVER	helix-deps.yaml	rc=1, offender named
GITREF		rc=1, offender named

Class	Host file	Result
	_tools/od/ generate.sh	
GPU	_tools/parse-chrome- translations.js	rc=1, offender named
SHEBANG	_tools/gen/build.sh (line 1)	rc=1, offender named

After removing every probe, the mirror returns to its exact baseline — scanned 174 · 87 baselined · 17 allowed · rc=0 — and the live tree does the same. `bash scripts/audit-hardcoded-paths.sh` remains rc=0 throughout.

7. The gate is free of what it polices

Verified by inspection, not asserted:

Property	Evidence
Portable interpreter	<code>#!/usr/bin/env bash</code>
Root derived, never literal	<code>ROOT="\$(cd -- "\${dirname -- "\${BASH_SOURCE[0]}")/.." && pwd)"</code>
No GNU-only tool flags in executable position	zero matches for <code>sed -i</code> , <code>readlink -f</code> , <code>stat -c</code> , <code>date -d</code> , <code>grep -P</code> , <code>xargs -r</code> , <code>sort -V</code> , <code>mktemp -p</code> , <code>du -b</code> , <code>cp --parents</code> outside its own pattern strings
POSIX <code>awk</code> only	no <code>gensub</code> , no <code>ENDFILE</code> , no <code>asort</code> , no <code>{n,m}</code> interval quantifiers (<code>mawk 1.3.3</code> does not support them)
No service or package manager invoked	zero
Prerequisites probed, not assumed	<code>for _bin in git awk grep; do command -v "\$_bin" ... → rc=2</code>
Temp dir respects the environment	<code>TMPDIR_SAFE="\${TMPDIR:-/tmp}"</code> , <code>mktemp -d</code> with a template, <code>trap ... EXIT INT TERM</code>
No absolute machine paths	zero

Property	Evidence
Colour respects NO_COLOR and non-TTY output	[-t 1] && [-z "\$ {NO_COLOR:-}"]

It allow-lists **itself** (and its sibling `scripts/audit-hardcoded-paths.sh`) with a stated reason: a detector’s class patterns must literally contain every token it searches for. That is the same precedent as `.hardcoded-paths-allow` exempting `scripts/audit-hardcoded-paths.sh`, and probe D3 proves the exemption is path-scoped — the identical literal in any other file is still caught.

8. Scope boundary

This audit **did not fix anything**. Fixing is a separate decision, and several of the affected files were being edited by other agents while the sweep ran; a concurrent rewrite would have collided. The 85 verified defects are carried as BASELINE entries so the gate can be adopted immediately and catch the 86th.

9. GNU/BSD portability claims — measured, 2026-09-01

Until this section was written, every BSD-side portability claim in this tree rested on **simulation**. The stated reason was that “no BSD/macOS host is reachable”. That reason was over-broad, and this section replaces it with what was actually established. **Verdicts are CONFIRMED / REFUTED / STILL-UNTESTED, and nothing was upgraded to CONFIRMED by relabelling.**

9.1 Mechanisms attempted, and what each actually yielded

#	Mechanism	Outcome
1	BSD userland in a container	PARTIAL — image yes, execution no. podman pull --os freebsd --arch amd64 docker.io/freebsd/ freebsd-runtime:14.2

#	Mechanism	Outcome
2	Build the real BSD utilities from source	succeeds; without --os freebsd podman refuses (“no image found ... OS linux”). The rootfs exports and reads fine: /bin/date, /usr/bin/sed, /bin/sh, /libexec/ld-elf.so.1. The binaries cannot run on this Linux kernel. They are ELF “for FreeBSD 14.2”, so a direct exec fails on the missing interpreter, and <pre>podman run --rootfs <fbbsdroot> /bin/date -r 10000000000 —</pre> which supplies the interpreter — dies with rc 139 (SIGSEGV) at the first FreeBSD syscall. Linux has no FreeBSD ABI layer and no qemu-bsd-user is packaged here. The rootfs is still evidentially useful for <i>filesystem</i> facts (see §9.3, env/bash/hashing). SUCCESS — this is what produced the measurements. FreeBSD 14.2 usr.bin/stat, bin/date and usr.bin/sed compile against glibc and run natively. Recipe in §9.2.

#	Mechanism	Outcome
3	Package-manager BSD variants	PARTIAL — no BSD, but two non-GNU implementations. This host (ALT Linux 11, apt-rpm) has no sbase, 9base, bsdmainutils, heirloom-tools or libbsd package. It does have busybox 1.37.0 and toybox 0.8.13; both were fetched as RPMs and unpacked into a scratch prefix — no system install, no su do (there is no passwordless sudo on this host). Neither is BSD — both follow GNU spellings — but toybox produced the sharpest single finding in this area (§9.3). NOT REACHED as a running host. No macOS machine is reachable. What <i>was</i> obtained is the genuine Apple source (apple-oss-distributions/shell_cmds, file_cmds, text_cmds), which is the same code the shipped binaries are built from, and which settled the date -d question historically as well as currently.
4	macOS itself	

9.2 The build recipe (mechanism 2)

Reproducible from any host with curl and a C compiler; no network access is needed at gate time, only at build time.

```
B=https://raw.githubusercontent.com/freebsd/freebsd-src/releng/14.2
mkdir -p bsd/src && cd bsd/src
for f in usr.bin/stat/stat.c bin/date/date.c bin/date/vary.c bin/date/vary.h \
    usr.bin/sed/main.c usr.bin/sed/compile.c usr.bin/sed/misc.c \
    usr.bin/sed/process.c usr.bin/sed/defs.h usr.bin/sed/extern.h \
    lib/libc/string/strmode.c contrib/libc-pwcache/pwcache.c \
    contrib/libc-pwcache/pwcache.h; do
    mkdir -p "$(dirname "$f")" && curl -sS -o "$f" "$B/$f"
done
# then compile with a prelude supplying the FreeBSD-only libc interfaces glibc
# lacks (SPECNAMELEN, fhandle_t/fhstat, fdevname_r, st_*timespec, ishexnumber,
# errc, setpassent/setgrouper, __dead2/__printflike/nitems, getprogname):
gcc -DHAVE_CONFIG_H=1 -D_GNU_SOURCE -I. -include bsdcompat.h -o bsdstat \
    usr.bin/stat/stat.c lib/libc/string/strmode.c contrib/libc-pwcache/pwcache.c
gcc -D_GNU_SOURCE -I. -include bsdcompat.h -o bsddate bin/date/date.c bin/date/vary.c
gcc -D_GNU_SOURCE -I. -include bsdcompat.h -o bsdsed usr.bin/sed/*.c
```

Fidelity, stated honestly. The option parsing, format-string parsing, usage()/exit behaviour and the stat(2)/strftime(3) calls are the vendor's own code, unmodified — and those are precisely the code paths every claim below turns on. The shims cover the -H filehandle mode, the stdin-device-name mode, device major/minor decoding, and the timespec member spelling: **none of them is on any path under test.** Sanity cross-check on this host: the built bsdstat agrees with GNU stat (-f %Lp → 754, -f %m → the identical epoch), and the built bsddate -u -r 1000000000 +%Y agrees with GNU date -u -d @1000000000 +%Y. **This is not a measurement on a running BSD kernel**, and nothing here claims it is; it is a measurement of the genuine BSD implementation's logic.

Point the gates at the results to convert their simulations into measurements:

```
VASIC_BSD_STAT=/path/to/bsdstat bash scripts/verify-check-registry.sh --prove-filemode
VASIC_BSD_STAT=/path/to/bsdstat bash scripts/verify-manifest-pins.sh --prove-filemode
VASIC_BSD_STAT=/path/to/bsdstat bash _tools/pdf/build-pdfs.sh --prove-mtime
VASIC_BSD_DATE=/path/to/bsddate bash _tools/gen/build.sh --prove-buildyear
```

Each gate **probes the binary for the BSD contract before believing the label**, so a GNU binary passed in by mistake is refused and the run falls back to the simulation, saying so. Verified: passing the host's GNU stat produces

A2 bsd-binary ... did not answer to the BSD contract ... using the stub.

9.3 Per-claim verdicts

Claim	Verdict	Evidence
BSD stat -c '%a' errors cleanly with NO stdout , so an \ \ fallback <i>replaces</i> rather than appends	CONFIRMED	bsdstat -c '%a' <mode-754 file> → stdout 0 bytes , rc 1 , stderr invalid option -- 'c' + usage. Source-level corroboration on both platforms: FreeBSD usr.bin/stat option string "f:FHlLnqrst:x"; Apple file_cmds stat.c "f:FHlLnqrst:x" / "f:FlLnqrst:x". No c in either.
stat -f %m on GNU parses %m as a filename, prints the filesystem report to stdout, exits 1 — so a BSD-first \ \ \ chain appends to garbage	CONFIRMED (GNU) , and the BSD half now measured too	GNU stat -f %m → 233 bytes on stdout, rc 1. BSD stat -f %m → 1788279584, rc 0 , empty stderr. The two fail asymmetrically : the GNU-only spelling fails <i>cleanly</i> on BSD,

Claim	Verdict	Evidence
A third implementation makes output-validation mandatory	NEW FINDING	the BSD-only spelling fails <i>dirtyly</i> on GNU. toybox 0.8.13: stat -f %Lp <file> writes 215 bytes of filesystem report to stdout and <i>exits 0</i>. An exit-status test does not merely mis-handle that — it never fires. busybox 1.37.0 behaves like GNU (199 bytes, rc 1). A new assertion A5 was added to both <code>_file_mode</code> gates for this case. Same working directory, containing a file named 1000000000 whose mtime year is 1999: GNU <code>date -u -r 1000000000 +%Y → 1999</code>; BSD <code>date -u -r 1000000000 +%Y → 2001</code>. Nuance worth keeping: BSD -r accepts <i>either</i> — <code>strtoq</code> first, and only a non-fully-numeric argument is retried as a pathname (<code>bsddate -u -r notanumber +%Y → 1999</code>). That is why the known-answer oracle, not a four-digit shape test, is what makes the dispatch safe.
date -r: GNU --reference=FILE vs BSD epoch-seconds , genuinely flipping when a file named with the epoch exists in the cwd	CONFIRMED	

Claim	Verdict	Evidence
BSD/macOS -d “sets the kernel daylight-saving flag rather than parsing a date”	REFUTED for every BSD shipping now; TRUE historically	FreeBSD 14.2 date -u -d @10000000000 +%Y → invalid option -- 'd', usage on stderr, empty stdout, rc 1. Its option string is "f:I::jnRr:uv:z:" — no d. Current macOS is identical (apple-oss-distributions/shell_cmds main, same string). The DST reading was real up to and including shell_cmds-216.60.1 ("d:f:jnRr:t:uv:") and is gone by shell_cmds-302.60.2. The B4 simulation in _tools/gen/build.sh is therefore kept and relabelled as the <i>historical macOS</i> contract — machines presenting it are still in service — and a separate B5 measures the current contract. Note the modern behaviour could not replace B4: -d returning empty is rejected even by a shape test, so it cannot make B4's point.
sed -i: GNU takes no argument, BSD requires a backup suffix; the tree	CONFIRMED, in both directions	BSD sed given the GNU spelling -i 's/alpha/A/' file consumes the script as the backup suffix,

Claim	Verdict	Evidence
avoids it via temp-file editing		then treats the filename as the script: invalid command code <code>., rc 1</code>, file unchanged, no stray backup written. BSD-correct <code>-i ''</code> <code>'s/.../.../'</code> file $\rightarrow$ rc 0, edit applied. Conversely GNU sed given <code>-i '' script</code> file $\rightarrow$ can't read <code>s/alpha/A/</code>, rc 2. Option strings: FreeBSD <code>"EI:ae:f:i:lnru"</code>, Apple <code>"EHI:ae:f:i:lnru"</code> — <code>i:</code> requires an argument in both. The <code>_sed_i()</code> temp-file helper in <code>scripts/verify-check-registry.sh</code> and <code>scripts/verify-manifest-pins.sh</code> uses neither spelling and is unaffected. <code>shasum -a 256</code> and <code>sha256sum</code> produce byte-identical output on this host (Digest::SHA 6.04, the same Perl script macOS ships), and <code>sha256_file()</code> returns the correct digest with only shasum on PATH — the macOS case — as well as with <code>sha256sum</code>. The correction: the
sha256sum vs shasum -a 256 in <code>workshop/scripts/_portable.sh</code>	CONFIRMED, with one correction to the rationale	

Claim	Verdict	Evidence
#!/usr/bin/env bash finds bash where BSD/macOS puts it	CONFIRMED for FreeBSD by direct filesystem measurement	comment implies BSD lacks sha256sum. FreeBSD base ships /sbin/sha256sum (one of 24 hardlinks to md5(1), per sbin/md5/Makefile), it emulates the GNU CLI when invoked under that name, and it parses with getopt_long, so the -- in sha256sum -- "\$f" is handled. The claim is true of macOS, not of BSD generally. Behaviour is correct either way; only the stated reason is narrower than written. The genuine FreeBSD 14.2 rootfs has /usr/bin/env (14944 bytes) and no bash anywhere in base — bash is a package installed to /usr/local/bin/bash, which is on the default PATH. So #!/usr/bin/env bash is the correct portable form and #!/bin/bash would fail outright on FreeBSD. For macOS the same form works (/usr/bin/env plus /bin/bash), but that half is source/documentation-

Claim	Verdict	Evidence
		level, not measured here — see §9.4.

9.4 What is still NOT measured, and stays labelled that way

- **No running BSD or macOS kernel was reached.** Everything above is either the genuine vendor source compiled and run on Linux, or a direct read of a genuine FreeBSD rootfs. A kernel-level difference — anything where the syscall, not the utility, decides the answer — would not be caught by this method. None of the claims above turns on one, but the boundary is real and is not papered over.
- **macOS filesystem layout** (`/usr/bin/env`, `/bin/bash`) is asserted from Apple’s shipped source and documentation, not measured on a Mac.
- **A2 in both `_file_mode` gates remains a SIMULATION by default.** It becomes a measurement only when `VASIC_BSD_STAT` is supplied, and the gate prints which of the two it actually did. It is never relabelled without the evidence.
- **B4 in `_tools/gen/build.sh` remains a SIMULATION** — deliberately, because what it simulates (historical macOS `-d`) cannot be measured from any current source. It is now labelled as historical rather than as “the BSD contract”.

9.5 Two corrections to previously recorded measurements

Both are withdrawals, not restatements (§11.4.6).

1. **`scripts/verify-check-registry.sh` attributed a garbage-output measurement to the wrong `||` ordering.** It recorded that the GNU-first chain `stat -c %a f || stat -f %Lp f || printf '?'` produced “220 bytes of filesystem statistics on stdout ... 221 bytes” once the fallback appended. **That does not reproduce.** Re-measured on this host (GNU coreutils 9.7, mode-754 file): GNU-first → 754, **3 bytes**; BSD-first → **231 bytes** of filesystem report; GNU-first with the file absent → **?, 1 byte**. The garbage belongs to the **BSD-first** ordering — which is the ordering `portable_mtime` in `_tools/pdf/build-pdfs.sh` historically had. `_file_mode` has no prior form in git history that could have produced the recorded number. The conclusion drawn from it was and is correct; the ordering it was pinned to was not.

2. **“No BSD userland, no BSD container image, no busybox, no toybox” is withdrawn.** busybox and toybox are both packaged for this host; a FreeBSD 14.2 container image pulls; and the vendor source builds. The accurate, narrower statement is the one now in the code: a FreeBSD **userland cannot execute** on this Linux kernel (rc 139, SIGSEGV).

9.6 Two dated comments found while measuring — reported, not edited

Neither is a behavioural defect; both are inaccurate *rationales* in workshop/scripts/_portable.sh, which another agent owns. Recorded here so the owner can decide.

- *“GNU split -d gives numeric suffixes; BSD split has no -d and errors out.”* **Both current FreeBSD and current macOS DO have split -d:** FreeBSD 14.2 usr.bin/split and Apple text_cmds/split share the option string "0::1::2::3::4::5::6::7::8::9::a:b:cdl:n:p:", and FreeBSD's has case 'd': /* Decimal suffix */. The code is unaffected because split_numeric() **probes the capability at run time** rather than branching on platform — which is exactly why it survives the rationale being dated.
- *“macOS ships shasum -a 256 and no sha256sum at all”* — true of macOS, but the surrounding framing reads as BSD-wide, and **FreeBSD base does ship sha256sum**. See §9.3.